

Optic-SpaceNet 光计算迁移实验记录

三个 CNN 模型向 Gazelle 光计算硬件 (8×2 光学矩阵乘法器) 迁移的完整实验轨迹

目录

- [实验目标与硬件约束](#)
- [模型架构演进](#)
- [完整实验路线图](#)
- [Phase 0: FP32 基准训练](#)
- [Phase 1: QAT 微调 \(FP32→int4\)](#)
- [Phase 2: 从零 QAT 训练](#)
- [Phase 3: LSQ + 混合精度](#)
- [Phase 4: STE + 噪声注入 + 非对称量化](#)
- [Phase 5: Mixed Precision \(Conv=int4, Linear=fp32\)](#)
- [Phase 6: Gazelle 硬件匹配训练](#)
- [光计算容器迁移](#)
- [关键 Bug 记录](#)
- [精度演进总表](#)
- [文件清单](#)
- [后续方向](#)

1. 实验目标与硬件约束

1.1 目标

将三个 EuroSAT (10 类遥感图像, 64×64) CNN 模型迁移到 Gazelle 光计算硬件上运行, 最大化 int4/int8 量化精度, 保持硬件利用率 >95%。

1.2 Gazelle 硬件参数 (2026-07-09 逆向分析确认)

参数	值	来源
物理 tile	8×2 (k=8, n=2)	GAZELLE_ARCHITECTURE.md
原生激活精度	8-bit	模型名 8a8w12o

参数	值	来源
原生权重精度	8-bit	同上
输出精度	12-bit	—
DAC ENOB	7.5 bits	calibration_params.json
TIA noise MSE	2.85×10^{-7}	同上
硬件线性度	99.4% (相对误差 0.6%)	behavioral_char.json
对齐要求	im2col 展平长度被 8 整除	tile k=8

1.3 精度目标

模型	FP32 基准	int4 目标	int8 目标
Model 1 (VGG)	97.17%	≥ 95%	≥ 96%
Model 2 (SN V1)	90.15%	≥ 88%	≥ 90%
Model 3 (SN V2 KD)	91.44%	≥ 89%	≥ 91%

2. 模型架构演进

2.1 三个模型

	Model 1	Model 2	Model 3
名称	Baseline VGG	OpticSpaceNet V1	OpticSpaceNet V2
设计思路	标准 CNN 基线	硬件对齐 (2×2 conv)	硬件对齐 + KD
Conv 层	6× Conv2d (3×3)	4× Conv2d (1×1/2×2)	同 Model 2
Linear 层	2×	2×	2×
参数量	~2.39M	~268K	~268K
训练方式	标准分类	标准分类	KD (ResNet-18, 97.83%)
硬件对齐率	99.8% (首层 84.4%)	99.6% (stem 37.5%)	同 Model 2

2.2 架构变体演进

Phase 0-3 (原始):
Model 1: Sequential blocks, bias=True, 无 BN

Model 2/3: Sequential blocks, bias=False (Conv), bias=True (Linear)

Phase 4+ (新版, 匹配光计算硬件):

Model 1: Flat arch (conv1_1/bn1_1/...), bias=False (所有层)

Model 2/3: Sequential + BN, bias=False (所有层)

关键变化:

- bias=False: 光计算硬件不支持 bias 加法
- BN 保留: float32 运行, 稳定 QAT 训练
- Flat arch: 首层可独立控制 FP32/QAT

3. 完整实验路线图

Phase 0: FP32 基准训练 (2026-07-05)

└─ Model 1: 97.17% ✓ | Model 2: 90.15% ✓ | Model 3: 91.44% ✓

Phase 1: QAT 微调 (FP32→int4) (2026-07-05)

└─ Model 1: 85.91% ✗ | Model 2: 73.63% ✗ | Model 3: 73.22% ✗

└─ 结论: 微调路线完全失败

Phase 2: 从零 QAT 训练 (2026-07-05 ~ 06)

└─ Model 1: 91.17% △ | Model 2: 81.20% △ | Model 3: 83.26% △

└─ 结论: 有效但差距仍大 (6-9%)

Phase 3: LSQ + 混合精度 (2026-07-06)

└─ 旧版 LSQ+ (v2): 61.72% ✗ (代码 bug, 后于 Phase 6 修复至 92.80%)

└─ 结论: STE 方案被选定为主要方向; LSQ+ 修复后成为并行方向

Phase 4: STE + 非对称量化 + 噪声 (2026-07-06 ~ 07)

└─ Model 1: 96.46% int4 ✓ (optic_qat_v3, flat arch + BN)

└─ Model 2: 74.35% int4 ✗ (bug: Conv QAT 全关)

└─ Model 3: 78.26% int4 ✗ (bug: Conv QAT 全关)

Phase 5: Mixed Precision (Conv=int4, Linear=fp32) (2026-07-07)

└─ Model 1: 98.26% int4 ★★ | Model 2: 91.26% int4 ★

└─ Model 3: 91.13% int4 ★

Phase 6: Gazelle 硬件匹配训练 (2026-07-09 ~ 10)

└─ Model 2 v2 (int4, QAT 全开): 91.06% ★ 超越 FP32 基准!

└─ Model 2 v3 (int8, Gazelle 噪声): 93.11% ★★ STE 最佳!

└─ Model 2 LSQ+ (int8, 可学习 scale/zp): 92.80% ★★ LSQ+ 最佳!

└─ Model 3 v2 (KD+int4, QAT 全开): 91.50% ★ 超 FP32 KD 基准!

容器迁移 (2026-07-09)

- └─ optic_inference_phase4.py, optic_inference_mixed.py
- └─ QAT eval 模式：精度与训练一致, Optic 模式：osimulator 硬件仿真

4. Phase 0: FP32 基准训练

	Model 1	Model 2	Model 3
脚本	model1_baseline.py	model2_spacenet_v1.py	model3_spacenet_v2.py
权重	baseline_vgg.pth	spacenet_v1.pth	spacenet_v2_distilled.pth
Epochs	60	80	100 + KD
最佳准确率	97.17%	90.15%	91.44%
教师准确率	—	—	97.83% (ResNet-18)
硬件对齐率	99.8%	96.3%	96.3%

结论: FP32 基准确立，Model 1 最强，Model 2/3 受限于参数量但有硬件对齐优势。

5. Phase 1: QAT 微调 (FP32→int4)

方案: 加载 FP32 权重 → BN 融合 → QAT 层替换 → 低 lr 微调 15-20 epochs

模型	FP32	QAT 微调	损失	判定
Model 1	97.17%	85.91%	-11.26%	X
Model 2	90.15%	73.63%	-16.52%	X
Model 3	91.44%	73.22%	-18.22%	X

根因: FP32 权重依赖精细精度，突然 int4 量化破坏特征。15-20 epoch + 低 lr 无法逃逸坏局部最小值。

6. Phase 2: 从零 QAT 训练

方案: 随机初始化 → epoch 1 起全程 int4 伪量化 → 模型从未见过 float32

模型	FP32 基准	Phase 1	Phase 2	vs Phase 1	与 FP32 差距
Model 1	97.17%	85.91%	91.17%	+5.26%	-6.00%
Model 2	90.15%	73.63%	81.20%	+7.57%	-8.95%
Model 3	91.44%	73.23%	83.26%	+10.03%	-8.18%

结论: 从零 QAT 比微调提升 5-10%，验证了"从零学 int4 兼容特征"策略。但仍有 6-9% 差距。
问题: 动态 scale 不稳定、首/末层信息损失、Model 1 过拟合、Model 2/3 收敛慢。

7. Phase 3: LSQ + 混合精度

方案: LSQ 可学习 scale + 首层/末层 FP32

模型	模式	结果	问题
Model 1	LSQ+	61.72%	LSQ+ input_scale 初始化错误 + uint4 激活损失大
Model 1	STE	98.07% (训练中 FP32 模式最佳)	成功, 但 int4 eval 仅 96.46%

结论: 旧版 LSQ+ 因代码 bug (死参数 + uint4 瓶颈) 仅 61.72%。STE 被选定为主要方向。
后于 Phase 6 (2026-07-10) 修复 LSQ+ (int8 + 可学习 scale 真正参与前向), 达到 92.80%, 与 STE 并列为主要方向。

8. Phase 4: STE + 噪声注入 + 非对称量化

方案设计

- QAT 模块: optic_qat_v2.py (初版), optic_qat_v3.py (修复版)
- 量化: 激活 uint4 [0,15] → 修复为 int8; 权重 int4 [-8,7]
- 噪声: STE 训练时向权重注入高斯噪声 (std=0.05scale → 0.02scale)
- 架构: bias=False (匹配光硬件), BN 保留 (float32)
- 训练器: train_phase4_runner.py (Phase4Trainer)

8.1 原始版 (optic_qat_v2, Sequential arch)

模型	脚本	Int4	Float32	问题
Model 1 LSQ+	<code>model1_baseline_phase4.py --mode lsqplus</code>	61.72%	—	LSQ+ 有 bug, 后修复至 92.80%
Model 2 STE	<code>model2_spacenet_v1_phase4.py</code>	—	—	未完成 (训练中 bug)

Bug: `first_layer_fp32=True` 在 Sequential 架构中把所有 Conv 层都关了 (QAT Conv: 0)

8.2 修复版 (optic_qat_v3, flat+BN arch)

模型	Int4 (eval)	Float32	训练最佳 (FP32 模式)	备注
Model 1 STE	96.46%	98.06%	98.07%	✓ 成功, 量化损失仅 1.6%
Model 2 STE	74.35%	92.81%	92.87%	X Conv QAT 全关 (bug 未修复)
Model 3 KD STE	78.26%	93.15%	93.22%	X Conv QAT 全关 (bug 未修复)

关键发现: Model 2/3 的 QAT Conv: 0 — 训练时 Conv 在 FP32 模式, eval 时 `enable_qat()` 才打开。模型从未在训练中见过 int4 Conv 量化。

9. Phase 5: Mixed Precision (Conv=int4, Linear=fp32)

方案

- **QAT 模块:** `optic_qat_v3.py` (Conv→int4 QAT, Linear→fp32)
- **训练器:** `train_mixed_runner.py` (MixedPrecisionTrainer)
- **策略:** Conv 在光计算 (int4), Linear 在电计算 (fp32)
- **架构:** Flat+BN (Model 1), Sequential+BN (Model 2/3), Conv=bias=False, Linear=bias=True

结果

模型	脚本	Int4	Float32	FP32 基准	判定
Model 1	<code>model1_baseline_mixed.py</code>	98.26%	97.91%	97.17%	★★ 超越 FP32!

模型	脚本	Int4	Float32	FP32 基准	判定
Model 2	model2_spacenet_v1_mixed.py	91.26%	84.33%	90.15%	★ 超越 FP32!
Model 3 KD	model3_spacenet_v2_mixed.py	91.13%	86.33%	91.44%	★ 接近 FP32 KD

关键发现: Mixed 策略让所有 Conv 层 QAT 全开（无 first_layer_fp32 bug），模型真正学会了 int4 量化。Model 1/2 的 int4 精度甚至超过了 FP32 基准（QAT 正则化效应）。

10. Phase 6: Gazelle 硬件匹配训练

10.1 硬件逆向分析 (2026-07-09)

基于 osimulator/GAZELLE_ARCHITECTURE.md 逆向报告的关键发现:

发现	影响
硬件线性度 99.4% (误差 0.6%)	硬件几乎理想，量化精度是唯一瓶颈
原生精度 8a8w12o	硬件支持 int8 权重！我们一直用 int4 太保守
DAC ENOB= 7.5 , TIA noise= 5.3e-4	训练噪声 std=0.02*scale 是实际的 12 倍
tile 8×2	Model 2/3 的 2×2 conv (patch=32/64) 完美对齐 8 的倍数

10.2 修复方案

Phase 4 v2 (修复 Conv QAT 全关 bug):

- 使用 optic_qat_v3 的 prepare_model_v3（无 first_layer_fp32 参数）
- Conv+Linear 全 int4 QAT, bias=False
- 脚本: model2_spacenet_v1_phase4_v2.py , model3_spacenet_v2_phase4_v2.py

Phase 4 v3 (int8 权重 + Gazelle 硬件噪声):

- 使用新模块 optic_qat_v4.py
- int8 权重匹配硬件原生精度 (256 级 vs int4 的 16 级)
- GazelleNoiseInjector: DAC ENOB=7.5 + TIA noise
- 首层 stem FP32 (对齐率仅 37.5%, 电计算更划算)

- 其余 Conv+Linear 全 int8 QAT
- 脚本: `model2_spacenet_v1_phase4_v3.py`

10.3 结果

#	脚本	模型	配置	Int 精度	FP32 精度	耗时
1	<code>model2_..._phase4_v2.py</code>	Model 2	int4, QAT 全开	91.06%	87.54%	75min
2	<code>model3_..._phase4_v2.py</code>	Model 3	int4+KD, QAT 全开	91.50%	80.98%	119min
3	<code>model2_..._phase4_v3.py</code>	Model 2	int8+Gazelle, 修复	93.11%	93.02%	81min
4	<code>model2_spacenet_v1_lsqr.py</code>	Model 2	LSQ+ int8, 修复	92.80%	62.52%	89min

10.4 LSQ+ 修复版 (2026-07-10)

旧版 LSQ+ 的 Bug (`optic_qat_v2`):

- `out_scale / out_zp` 声明但从未参与前向 → 死参数, 无梯度
- `in_scale` 同样未使用, 输入量化用的仍是动态 `fake_quantize_uint4`
- 激活 uint4 (16 级) → 信息瓶颈
- 内部 ReLU + 输出重量化 → 干扰模型架构
- 无 BN → 激活分布不稳定

新版 LSQ+ (`optic_qat_lsqr.py`) 修复:

- `in_scale / in_zp` 通过 `lsqr_quantize()` 真正参与前向 + LSQ 梯度
- int8 激活 (256 级)
- 无内部 ReLU, 无输出重量化
- BN 保留
- STE warmup (10 epoch) → LSQ+ 切换, 避免初期不稳定
- LSQ 梯度 `sum_dims` 偏移量修复 (`x.dim() > scale.dim()` 时维度对齐)

结果: LSQ+ **92.80%**, 仅比 STE int8 (93.11%) 低 0.31%, 比旧版 LSQ+ (61.72%) 提升 **+31.08%**。

Float32 模式极低 (62.52%): LSQ+ 可学习 scale/zp 将权重推向极端值专门适配 int8 量化, 去掉量化后权重无法正常工作。这是 LSQ+ 固有特性, 不影响部署。

10.5 分析

- **v2 int4 (91.06%):** 修复后比旧版 (74.35%) 提升 **+16.71%**, 超过 FP32 基准 (90.15%)
- **v3 int8 (93.11%):** 匹配硬件原生 8-bit, 量化损失仅 0.09%, **比 FP32 基准高 2.96%**
- **LSQ+ int8 (92.80%):** 可学习 scale/zp, 比 STE 仅低 0.31%, 优势是 scale 可直接导出为硬件配置
- **int4 vs int8 vs LSQ+:** int8 大幅优于 int4; LSQ+ 接近 STE 但提供可部署的硬件参数
- **QAT 正则化效应:** int4/int8 精度 > float32 模式精度; LSQ+ 的 FP32 模式极低是因为权重过度特化

10.5 v3 开发中的 Bug

optic_qat_v4._convert_to_v4 的 _first 标志使用 Python 不可变 bool 参数, 嵌套递归时子级修改不传播到父级, 导致所有 Sequential 块内的 Conv 都被当作"首层"关闭 QAT。**已修复:** 改用单元素列表 [_first] 传递可变引用。

11. 光计算容器迁移

11.1 容器代码

文件	用途	模式
optic_layers.py	光计算核心库 (OpticalEngine, OpticConv2d, OpticLinear)	推理
optic_inference.py	FP32 基准模型容器 (Part A)	Native + Optic
noise_robustness.py	FP32 模型噪声鲁棒性 (Part B)	噪声扫描
noise_robustness_v2.py	int4 模型噪声鲁棒性	QAT eval
optic_inference_phase4.py	Phase 4 模型容器 (NEW)	QAT(默认) + Optic(--optic)
optic_inference_mixed.py	Mixed 模型容器 (NEW)	QAT(默认) + Optic(--optic)
optic_inference_int8.py	INT8 模型容器 (NEW, 含 MOPs 统计)	Optic(默认) + QAT(--qat) + MOPs-only(--mops-only)

11.1.1 INT8 容器 MOPs 统计 (Model 2 Phase4 v3)

光计算占比按每层 MAC 操作数 (Multiply-Accumulate Operations) 计算，输入尺寸 64×64×3:

层	类型	输入	输出	展平	对齐率	原始 MOPs	计算位置
stem.conv	Conv 3→8, 1×1	64×64	64×64	3→8	37.5%	0.098M	○ 电计算 (FP32)
stage1.conv	Conv 8→16, 2×2	64×64	32×32	32→32	100%	0.524M	● 光计算 (INT8)
stage2.conv	Conv 16→32, 2×2	16×16	8×8	64→64	100%	0.131M	● 光计算 (INT8)
stage3.conv	Conv 32→16, 1×1	8×8	8×8	32→32	100%	0.033M	● 光计算 (INT8)
fc1	Linear 1024→256	—	—	1024	100%	0.262M	● 光计算 (INT8)
fc2	Linear 256→10	—	—	256	100%	0.003M	● 光计算 (INT8)
合计						1.051M	

- 光计算 MOPs: 0.953M (stage1+stage2+stage3+fc1+fc2)
- 电子计算 MOPs: 0.098M (stem 首层)
- ★ 光计算占比: 90.65%
- 所有光计算层展平长度均为 8 的倍数，完美对齐 Gazelle 8×2 tile，无补零浪费
- stem 首层展平=3 对齐率仅 37.5%，保留电计算更高效

11.2 容器评估模式

Optic 模式 (默认) — 真实 osimulator 硬件仿真:

```
python optic_inference_int8.py                                # INT8 全量测试集 + MOPs 统计
python optic_inference_int8.py --quick 50                    # 快速测试 ~3min
python optic_inference_int8.py --mops-only                    # 仅 MOPs 统计
```

- build_optical_model → OpticConv2d/OpticLinear → osimulator

QAT 模式 (--qat) — PyTorch 伪量化交叉验证:

```
python optic_inference_int8.py --qat                          # QAT 伪量化 (容器外可用)
```

- build_optical_model() → OpticConv2d + OpticalEngine → osimulator

- 真实硬件仿真 (im2col 展开, 补零对齐, 物理噪声)
- 慢 (每 batch ~3-8 分钟), 建议 `--batch 1 --quick 5`

11.3 容器 osimulator 兼容性修复

问题	修复
<code>_matmul_real</code> 返回 Tensor 非 numpy	<code>isinstance(raw_result, torch.Tensor)</code> 兼容
输入量化 <code>signed=False</code> + <code>zero_point</code> 未补偿	反量化公式: <code>scale * result_int + in_zp * w_scale * col_sum_w</code>
<code>print_interval</code> 逐 batch 过于密集	默认每 10% 打印一次, Optic 模式保持逐 batch
默认 <code>batch=32</code> 产生 131K 行 im2col 矩阵	改为 <code>batch=1</code> , <code>--batch N</code> 可选

11.4 容器验证结果 (Phase4 QAT 模式, 全量 5400 张)

模型	训练 Int4/Int8	容器 Int4/Int8	误差
Model 1 Phase4 STE	96.46%	96.44%	-0.02% ✓
Model 2 Phase4 STE	74.35%	74.70%	+0.35% ✓
Model 3 Phase4 KD STE	78.26%	78.06%	-0.20% ✓

容器精度与训练完全一致。Model 2/3 偏低是因为旧版训练的 Conv QAT bug，不是容器问题。

11.5 INT8 容器 osimulator 真实硬件仿真 (2026-07-10)

背景: Phase 6 训练的 Model 2 v3 INT8 (93.11%) 需要在容器内通过真实 osimulator 验证。

开发过程:

阶段	描述	结果
初版部署	<code>optic_inference_int8.py</code> — Optic 模式默认, 含 MOPs 统计	84.43% X
Bug #6 发现	OpticConv2d/OpticLinear 内预量化 (signed int8) + <code>_matmul_real</code> 再量化 (unsigned uint8) = 双重量化	—
Bug #7 发现	<code>_matmul_fake</code> 硬编码 <code>quantize_int4</code> , 无视传入的 <code>input_bit=8 / weight_bit=8</code>	—

阶段	描述	结果
Bug #8 发现	build_optical_model 无条件将所有 Conv→OpticConv2d, stem (训练时 FP32) 也被转换	—
修复 v2	消除双重量化 + 修复 fake 引擎位宽 + stem 保留电计算	93.28% ★

最终结果:

```
python optic_inference_int8.py    # 5400 张独立测试集（与训练 val 零重叠）
```

指标	值
光计算准确率	93.28%
训练 QAT 参考	93.11% (训练 val set)
量化损失 (vs 训练)	+0.17% (略高于训练, 独立测试集差异)
总耗时	14841s (~4.1h)
光计算占比	90.65% (5/6 层在光计算)
硬件对齐率	99.6%
容器文件	optic_inference_int8.py
日志	log_optic_int8.md

MOPs 分布:

层	MOPs	位置
stem (Conv 3→8, 1×1)	0.098M (9.3%)	电计算 FP32
stage1 (Conv 8→16, 2×2)	0.524M (49.9%)	光计算 INT8
stage2 (Conv 16→32, 2×2)	0.131M (12.5%)	光计算 INT8
stage3 (Conv 32→16, 1×1)	0.033M (3.1%)	光计算 INT8
fc1 (Linear 1024→256)	0.262M (24.9%)	光计算 INT8
fc2 (Linear 256→10)	0.003M (0.2%)	光计算 INT8

11.6 其余模型容器验证

基于 INT8 容器经验, 为其余模型创建了容器验证文件:

文件	模型	训练精度	Quick	全量 osimulator	光计算 占比
optic_inference_int4.py	Model 2 v2 INT4	91.06%	~88% (quick 50)	87.94% (全 量 5400)	90.65%
optic_inference_lsq.py	Model 2 LSQ+	92.80%	96.00% (quick 50)	92.76% (全 量 5400)	90.65%
optic_inference_kd.py	Model 3 v2 INT4	91.50%	83.50% (quick 200)	84.33% (全 量 5400)	90.65%
optic_inference_kd.py	Model 3 v3 INT8+KD	92.35%	96.00% (quick 50)	93.26% (全 量 5400)	90.65%
optic_inference_mixed_model1.py	Model 1 Mixed	98.26%	100% (quick 50/100)	不可行 (⚠ ~9天)	98.67%

11.7 Model 3 v3 int8+KD — 训练与 osimulator 验证 (2026-07-12) ★

训练:

```
python model3_spacenet_v2_phase4_v3.py # 100 epochs, ~1.8h
```

指标	值
训练配置	stem FP32 + Conv/Linear int8 + Gazelle 噪声 + KD (T=4.0, $\alpha=0.7$)
Int8 QAT 准确率	92.35%
Float32 准确率	92.31%
量化损失	-0.04% (几乎无损!)
权重文件	spacenet_v2_phase4_v3_int8.pth

osimulator Quick 50 验证:

```
python optic_inference_kd.py --weight spacenet_v2_phase4_v3_int8.pth --quick  
50
```

指标	值
光计算准确率	96.00% (50 张)
总耗时	147s (~3min)

osimulator 全量 5400 验证 (2026-07-12):

指标	值
光计算准确率	93.26% (5400 张独立测试集)
训练 QAT 参考	92.35% (val set)
量化损失 (vs 训练)	+0.91% (独立测试集, 正常波动)
总耗时	21784s (~6.0h)
引擎调用 / 总 MACs	27000 次 / 5.15e+09
光计算占比	90.65%

进度	准确率	进度	准确率
540/5400 (10%)	93.52%	3240/5400 (60%)	93.21%
1080/5400 (20%)	93.15%	3780/5400 (70%)	93.31%
1620/5400 (30%)	93.52%	4320/5400 (80%)	93.29%
2160/5400 (40%)	93.47%	4860/5400 (90%)	93.17%
2700/5400 (50%)	93.37%	5400/5400 (100%)	93.26%

全量 93.26% 与训练 92.35% 基本一致（独立测试集正常波动 +0.91%）。

对比 v2 int4:

	v2 int4 (旧)	v3 int8 (新)
训练精度	91.50%	92.35%
osimulator 全量	84.33%	93.26%
osimulator 损失	-7.17%	+0.91%
根因	int4→int8 三重不对齐	训练推理配置天然对齐

结论: v3 int8+KD 方案彻底解决了 int4 模型的 osimulator 推理难题。
核心是将训练配置 (stem FP32 + int8 权重 + Gazelle 噪声) 与 osimulator 推理路径

(keep_first_conv_electronic + 8a8w) 完全对齐。

训练脚本: model3_spacenet_v2_phase4_v3.py

推理脚本: optic_inference_kd.py --weight spacenet_v2_phase4_v3_int8.pth

11.8 Model 1 Mixed Quick 100 验证 (2026-07-12)

```
python optic_inference_mixed_model1.py --quick 100 # ~4.2h
```

指标	值
光计算准确率	100.00% (100 张全对)
训练参考	98.26% int4 Mixed
总耗时	14967s (~4.2h)
单张耗时	~150s

Quick 50 和 Quick 100 均达到 100% (150/150 全对), int4 Mixed 方案在 osimulator 上抽样完全无损。

11.9 LSQ+ 全量 osimulator 验证 (2026-07-11) ★

```
python optic_inference_lsq.py # 5400 张独立测试集
```

指标	值
光计算准确率	92.76%
训练 QAT 参考	92.80% (val set)
量化损失 (vs 训练)	-0.04% (几乎无损!)
总耗时	18118s (~5.0h)

LSQ+ 的 per-channel learned scales 使量化后的数据天然适合 osimulator 重量化, 是全系列模型中推理精度与训练精度最接近的。

11.10 INT4 容器全量 osimulator 验证 (2026-07-12) ⚠

```
python optic_inference_int4.py # 5400 张全量
```

```
python optic_inference_int4.py --qat # QAT 交叉验证
```

指标	值
光计算准确率 (全量 5400)	87.94%
QAT int4 (test set, 全量)	94.57%
QAT float32 (test set)	91.43%
训练 val 参考	91.06%
量化损失 vs QAT test	-6.63%
总耗时	20316s (~5.6h)

全量进度曲线:

540/5400 (10.0%) acc=85.74%	2160/5400 (40.0%) acc=87.41%
1080/5400 (20.0%) acc=87.22%	2700/5400 (50.0%) acc=88.00%
1620/5400 (30.0%) acc=87.47%	3240/5400 (60.0%) acc=87.96%
	3780/5400 (70.0%) acc=88.07%
4320/5400 (80.0%) acc=88.17%	4860/5400 (90.0%) acc=88.00%
5400/5400 (100.0%) acc=87.94%	

根因 (详见 §16):

- 1. int4→int8 权重量化网格不对齐 (scale=max/7 → max/127)
- 2. per-channel→per-tensor 激活量化退化
- 3. stem QAT→FP32 BN 统计量偏移

Model 1 速度限制

⚠ Model 1 的 MACs 是 Model 2 的 ~150 倍 (156.6M vs 1.05M/张), 全量 5400 张预计 ~9 天。

推荐策略:

```
python optic_inference_mixed_model1.py --qat # QAT 精度评估 (秒级全量)
python optic_inference_mixed_model1.py --quick 5 # Optic 硬件抽样 (~10min)
```

11.11 Model 1 Phase4 v3 int8 — 训练与 QAT 交叉验证 (2026-07-13) ⚠

⚠ 本节及全文「独立 test 集 / osimulator」数字受 Bug #11 (test⊂train 泄漏) 污染, 已作废; 下列精度均为 val 集 (模型选择指标, 5400 张, 未参与梯度训练), 是当前唯一有效的泛化数

字。

训练 (CPU, 各 ~6h):

```
python model1_baseline_phase4_v3.py --variant A # conv1_1 电计算, 光计算占比 97.7%
```

```
python model1_baseline_phase4_v3.py --variant B # conv1_1 + conv3_2 电计算, 光计算占比 73.6%
```

变体	电计算层 (FP32)	Int8 QAT (val)	Float32 (val)	量化损失	光计算 MOPs 占比	权重
A	conv1_1	98.15%	98.13%	-0.02%	97.74% (153.09M / 156.63M)	baseline_vgg_phase4_v3_int8
B	conv1_1 + conv3_2	98.02%	98.07%	+0.06%	73.64% (115.35M / 156.63M)	baseline_vgg_phase4_v3_int8

关键结论:

- **int8 量化无损:** 量化损失 $\pm 0.06\%$ (A -0.02% / B $+0.06\%$), 优于 int4 STE (96.46%) 且超过 FP32 基准 97.17%。验证 int8 权重 + Gazelle 噪声配方的精度优势。
- **A vs B 几乎打平** (98.15% vs 98.02%, $\Delta=0.13\%$): conv3_2 回退电计算几乎不付精度代价 (5400 张 val 上 $\pm 0.2\%$ 波动内不显著), 却换 osimulator 提速 ~24%。

QAT 交叉验证 (秒级):

```
python optic_inference_int8_model1.py --variant A --qat --batch 256
python optic_inference_int8_model1.py --variant B --qat --batch 256
```

- 验证通过 ✓: 权重加载、int8 量化无损、变体 B 的 conv3_2 正确还原电计算 (ElecMOPs 3.54M→41.29M)、MOPs 占比达标 (>50%)。
- **⚠ test 集 99.96% 作废:** 100% 落在训练集内 (Bug #11), 测的是记忆。真实泛化以 val 98.15%/98.02% 为准。
- 备注: 推理 `--qat` 路径 `noise=False` (干净伪量化上界), Gazelle 噪声在 `model.eval()` 下不触发, 故噪声不是 val/test 差值的来源 —— 差值完全来自数据泄漏。

osimulator 真机: Model 1 MACs 是 Model 2 的 ~150x (156.6M vs 1.05M/张), 全量 5400 张 ~9 天不可行; 待 Bug #11 修复重训后用 `--quick 50` 抽样。

12. 关键 Bug 记录

Bug #1: QAT eval 模式未施加量化

- `if self._qat_enabled and self.training: → model.eval()` 时跳过量化
- 修复: `if self._qat_enabled:`

Bug #2: first_layer_fp32 在 Sequential 中禁用所有 Conv

- `_first` 是不可变 bool, 嵌套递归不更新父级
- 影响: Phase 4 original, Phase 4 fixed Model 2/3, Phase 6 v3 (初版)
- 修复: Phase 4 v2 直接用新版 `prepare_model_v3` (无此参数); Phase 6 v3 使用 `[_first]` 列表

Bug #3: _matmul_real 输入 unsigned 量化 zero_point 未补偿

- `quantize_to_int(signed=False)` 产生 `in_zp ≠ 0`, 但反量化只用 `result * scale`
- 修复: `result = scale * result_int + in_zp * w_scale * col_sum_w`

Bug #4: osimulator 不接受负数输入索引

- `signed=True` 量化产生负值 → `osimulator index -105 out of bounds`
- 修复: 输入用 `uint4` (unsigned), 硬件只接受非负

Bug #5: LSQ+ 旧版 (optic_qat_v2) 多重 bug → 61.72%

- `out_scale / out_zp` 声明但从未参与前向 → 死参数无梯度
- `in_scale` 同样未使用, 输入量化仍用动态 `fake_quantize_uint4`
- 激活 `uint4` (16 级) → 信息瓶颈
- 内部 ReLU + 输出重量化 → 干扰模型架构
- 无 BN → 分布不稳定
- **Phase 6 修复:** 重写 `optic_qat_lsq.py`, `int8` + 真正可学习 `scale` + BN, 达到 92.80%

Bug #6: 光计算容器双重量化 (2026-07-10)

- `OpticConv2d.forward()` 先 `quantize_symmetric` (signed `int8`) 预量化
- 然后 `_matmul_real` 再 `quantize_to_int(signed=False)` (unsigned `uint8`) 二次量化
- 两次量化叠加 → 精度从 93% 跌至 84.43%

- 修复: `engine.use_real` 时跳过预量化, 传 `raw float`, `quantize_inputs=True`

Bug #7: `_matmul_fake` 硬编码 `int4` (2026-07-10)

- `_matmul_fake` 内始终调用 `quantize_int4()`, 无视 `input_bit/weight_bit` 参数
- 即使传入 `input_bit=8`, 模拟引擎仍按 `int4` 量化
- 修复: 改用 `quantize_symmetric(x, bits=input_bit)` 和 `quantize_symmetric(w, bits=weight_bit)`

Bug #8: `stem` 层被错误转为光计算 (2026-07-10)

- `build_optical_model` 无条件将所有 `Conv`→`OpticConv2d`
- 训练时 `stem` 是 `FP32` (`first_conv_fp32=True`, 对齐率仅 37.5%)
- 修复: `build_optical_model` 新增 `keep_first_conv_electronic=True`, 保留首个 `Conv2d` 不转换

Bug #9: `_matmul_real` `per-tensor` 权重量化 (2026-07-10)

- `quantize_to_int(weight_matrix, signed=True)` 对整个 `(k,n)` 矩阵用单一 `scale`
- QAT 训练用的是 `per-output-channel` 量化 (每个输出通道独立 `scale`)
- KD 模型通道间权重分布差异大, `per-tensor` 浪费精度 → 83.50%
- 修复: `_matmul_real` 改为 `per-channel` 权重量化, `w_scale` 从标量变为向量 `(n,)`
- 此修复对所有模型有益, 特别是 `int4` 和 `KD` 模型

Bug #10: `LSQ+ per-channel scale` 被通用路径破坏 (2026-07-10)

- `LSQ+` 的 `in_scale/in_zp/weight_scale/weight_zp` 是训练学出来的
- `build_optical_model` → `quantize_to_int` 重新计算 `scale` → 精度降至 15-60%
- `in_scale` 跨通道差异高达 6722x, `per-tensor` 近似完全失效
- 修复: Monkey-patch `LSQ` 层, 保留 `lsq_quantize` + 学到的 `scale/zp`, 量化后送 `osimulator`
- `LSQ` 量化后的粗粒度网格使 `_matmul_real` 再量化基本无损 → 96.00%

Bug #11: 独立测试集泄漏进训练集 (`test ⊂ train`) (2026-07-13) ⚠ ⚠

- 现象: Model 1 `int8 v3 QAT` 交叉验证在「独立 test 集」得 **99.96%**, 远高于训练 `val` **98.15%**, 且 ≈ 训练集准确率 99.81% —— 典型训练样本记忆特征。
- 根因: `load_test_data` (7 个推理脚本通用, `seed=42`) 取 `test = indices[val_size : val_size*2]`; 而 `load_eurosat_data` 训练用 `train = indices[val_size:]` → **test 整段落在 train 内, 实测 100% 重叠**。`load_test_data` 只断言 `testnval==0`, 漏查 `testntrain`。`optic_inference_int8.py:138` 注释甚至写明「test 来自训练集」, 作者误以为「未用于验证」即安全, 忽略了它用于了梯度更新。

- **影响 (全模型):** 三个训练脚本 (Model 1/2/3) 共用 `load_eurosat_data`, 故使用污染 test 段的推理脚本 (`int4/int4_v2/int8/int8_model1/kd/lsq`) 报的「独立测试集」数字均作废: Model 1 test 99.96%、Model 2 osimulator 93.28%、Model 3 osimulator 93.26%/Quick 96.00%、LSQ+ 92.76%、INT4 87.94%。**例外:** `optic_inference_mixed_model1.py` 用 val 段 (`indices[:sz]`) 评估, 不受影响 (Model 1 Mixed Quick 100=100% 为 val 基)。val 集 (`indices[val_size]`) 未参与训练, 一律有效: Model 1 int8 v3 98.15%、Model 2 v3 93.11%、Model 3 v3 92.35%。
- **修复 (keystone, 已应用 2026-07-13):** `train_phase4_runner.py:load_eurosat_data` 改 `train = indices[val_size*2:]` → 三分 **val(5400) / test(5400) / train(16200)**, 三者严格不相交 (已 empirical 验证: $\text{test} \cap \text{train} = \text{test} \cap \text{val} = \text{train} \cap \text{val} = 0$)。推理侧 `load_test_data` 无需改 (本就取 `indices[sz:2*sz]`, 修复后自动 disjoint)。
- **待办:** 现有权重均见过 test 图, 要拿干净 test 数必须用修复后的 split 重训。历史 osimulator 「独立测试集」数字需重训后复测。

13. 精度演进总表

Model 1 (Baseline VGG, ~2.39M params, FP32=97.17%)

Phase	方案	量化	Int 精度	vs FP32	状态
1	QAT 微调	int4	85.91%	-11.26%	X
2	从零 QAT	int4	91.17%	-6.00%	△
3	LSQ+ (旧版, bug)	int4	61.72%	-35.45%	X (后修复至 92.80%)
4	STE (修复)	int4	96.46%	-0.71%	✓
5	Mixed	Conv=int4, Linear=fp32	98.26%	+1.09%	★★
7 v3	Gazelle int8 (变体 A)	int8, conv1_1 FP32	98.15% (val)	+0.98%	★ (8a8w 对齐 osimulator)
7 v3	Gazelle int8 (变体 B)	int8, conv1_1+conv3_2 FP32	98.02% (val)	+0.85%	★ (提速 ~24%)

Model 1 结论: Mixed (98.26%) val 精度最高; **int8 v3 变体 A (val 98.15%)** 是硬件原生 8-bit 路径, 量化无损且与 osimulator 8a8w 天然对齐; 变体 B (val 98.02%) 牺牲精度 0.13% 换 osimulator 提速 ~24%。

⚠ 上表均为 **val 集** (模型选择指标); 历史所有「独立 test 集」数字因 Bug #11 作废。详见 §11.11 / §12。

Model 2 (SpaceNet V1, ~268K params, FP32=90.15%)

Phase	方案	量化	Int 精度	vs FP32	状态
1	QAT 微调	int4	73.63%	-16.52%	✗
2	从零 QAT	int4	81.20%	-8.95%	△
4	STE (bug)	int4	74.35%	-15.80%	✗
5	Mixed	Conv=int4, Linear=fp32	91.26%	+1.11%	★
6 v2	Phase4 修复	int4, QAT 全开	91.06%	+0.91%	★
6 v3	Gazelle 匹配	int8, 首层 FP32	93.11%	+2.96%	★★
容器 osimulator	真实光计算硬件仿真	int8, stem 电计算	93.28%	+3.13%	★★

Model 2 结论: v3 int8 (93.11%) 训练最佳; 容器 osimulator 真实硬件仿真 **93.28%** (独立测试集), 略超训练精度。从旧版 74.35% 到 93.28%, 提升 **+18.93%**。

Model 3 (SpaceNet V2 KD, ~268K params, FP32=91.44%)

Phase	方案	量化	Int 精度	vs FP32	状态
1	QAT 微调	int4	73.22%	-18.22%	✗
2	从零 KD+QAT	int4	83.26%	-8.18%	△
4	KD+STE (bug)	int4	78.26%	-13.18%	✗
5	KD+Mixed	Conv=int4, Linear=fp32	91.13%	-0.31%	★
6 v2	KD+Phase4 修复	int4, QAT 全开	91.50%	+0.06%	★
6 v3	KD+Gazelle 匹配	int8, stem FP32	92.35%	+0.91%	★★
容器 osimulator	真实光计算硬件仿真	int8, stem 电计算	93.26%	+1.82%	★★

Model 3 结论: v3 int8+KD (92.35%) 训练最佳 + osimulator 推理 **93.26%** (独立测试集)。Quick 50 = 96.00%。从 v2 int4 osimulator 84.33% 到 v3 int8 osimulator 93.26%, 提升 **+8.93%**。

总体最佳精度

模型	最佳方案	最佳 Int 精度	FP32 基准	提升
Model 1	Mixed (Conv=int4, Linear=fp32)	98.26%	97.17%	+1.09%
Model 1	容器 osimulator Quick 100	100.00% (抽样)	97.17%	+2.83%
Model 2	Phase4 v3 STE int8 + Gazelle	93.11%	90.15%	+2.96%
Model 2	容器 osimulator 真实硬件仿真	93.28% ★	90.15%	+3.13%
Model 2	Phase6 LSQ+ int8 (修复)	92.80%	90.15%	+2.65%
Model 3	Phase4 v2 int4 + KD	91.50%	91.44%	+0.06%
Model 3	Phase4 v3 int8+KD + Gazelle	92.35% ★	91.44%	+0.91%
Model 3	容器 osimulator 全量	93.26% ★	91.44%	+1.82%
Model 3	容器 osimulator (Quick 50)	96.00%	91.44%	+4.56%

⚠ **Bug #11 (test⊂train) 影响范围 (已审计全推理脚本):** 使用污染 test 段
(indices[sz:2*sz] ⊂旧 train) 的脚本 = int4 / int4_v2 / int8 / int8_model1 / kd / lsq —— 对应数字作废: Model 2 osimulator **93.28%**、Model 3 osimulator **93.26%** / Quick **96.00%**、LSQ+
92.76%、INT4 **87.94%**、Model 1 int8 v3 test **99.96%**。例外:
optic_inference_mixed_model1.py 用 val 段评估, 不受影响 (Model 1 Mixed Quick 100 = 100% 为 val 基, 仅含 model-selection peek)。val 集数字一律有效: Model 1 int8 v3 98.15%、Model 2 v3 93.11%、Model 3 v3 92.35%。split 已修复 (eurosat_split.py 单一数据源), 待重训后复测 osimulator 数。

14. 文件清单

训练脚本

文件	Phase	用途	产出权重
model1_baseline.py	0	FP32 训练	baseline_vgg.pth
model2_spacenet_v1.py	0	FP32 训练	spacenet_v1.pth
model3_spacenet_v2.py	0	FP32 KD 训练	spacenet_v2_distilled teacher_resnet18.pth
model1_baseline_qat.py	1	QAT 微调	baseline_vgg_qat.pth
model2_spacenet_v1_qat.py	1	QAT 微调	spacenet_v1_qat.pth
model3_spacenet_v2_qat.py	1	QAT 微调	spacenet_v2_qat.pth
model1_baseline_int4.py	2/3	从零 QAT/LSQ	baseline_vgg_int4.pth

文件	Phase	用途	产出权重
model2_spacenet_v1_int4.py	2/3	从零 QAT/LSQ	spacenet_v1_int4.pth
model3_spacenet_v2_int4.py	2/3	从零 KD+QAT/LSQ	spacenet_v2_int4.pth
model1_baseline_phase4.py	4	Phase4 STE/LSQ+	baseline_vgg_phase4_s_lsqplus.pth
model2_spacenet_v1_phase4.py	4	Phase4 (bug)	spacenet_v1_phase4_st
model3_spacenet_v2_phase4.py	4	Phase4 KD (bug)	spacenet_v2_phase4_st
model1_baseline_mixed.py	5	Mixed	baseline_vgg_mixed_st
model2_spacenet_v1_mixed.py	5	Mixed	spacenet_v1_mixed_ste
model3_spacenet_v2_mixed.py	5	Mixed KD	spacenet_v2_mixed_ste
model2_spacenet_v1_phase4_v2.py	6	Phase4 v2 int4 (修复)	spacenet_v1_phase4_v2
model3_spacenet_v2_phase4_v2.py	6	Phase4 v2 KD+int4 (修复)	spacenet_v2_phase4_v2
model3_spacenet_v2_phase4_v3.py	6	Phase4 v3 KD+int8+Gazelle (NEW)	spacenet_v2_phase4_v3
model2_spacenet_v1_phase4_v3.py	6	Phase4 v3 int8+Gazelle	spacenet_v1_phase4_v3

核心库

文件	版本	用途
optic_layers.py	v1	光计算推理核心 (OpticalEngine, OpticConv2d, 噪声注入器)
optic_qat.py	v1	QAT 初版: fake_int4, LSQ, QATConv2d, BN 融合
optic_qat_v2.py	v2	QAT Phase4 初版: uint4/int4 非对称, LSQ+, QATConv2d_v2
optic_qat_v3.py	v3	QAT 修复版: int8 激活, BN 保留, 无 first_layer_fp32
optic_qat_v4.py	v4	Gazelle 硬件匹配: int8 权重, GazelleNoiseInjector, 首层 FP32
optic_qat_lsq.py	v5	LSQ+ 修复版: int8 可学习 scale/zp, 无内部 ReLU, BN 保留
train_phase4_runner.py	—	Phase4 训练器 (Phase4Trainer)
train_mixed_runner.py	—	Mixed 训练器 (MixedPrecisionTrainer)

容器代码

文件	用途
optic_inference.py	FP32 基准模型容器
noise_robustness.py	FP32 噪声鲁棒性
noise_robustness_v2.py	int4 噪声鲁棒性
optic_inference_phase4.py	Phase4 模型容器 (QAT + Optic 双模式)
optic_inference_mixed.py	Mixed 模型容器 (QAT + Optic 双模式)
optic_inference_int8.py	INT8 模型容器 (已验证 93.28%)
optic_inference_int4.py	INT4 模型容器 (~90% quick)
optic_inference_lsq.py	LSQ+ 模型容器 (LSQ 专用路径, 96.00% quick)
optic_inference_kd.py	KD+INT4 模型容器 (83.5% 旧版, per-channel 修复后待重测)
optic_inference_mixed_model1.py	Model 1 Mixed 容器 (⚠ ~150s/张, 仅抽检)

文档

文件	用途
EXPERIMENTS.md	本文件: 完整实验记录
PHASE4_DESIGN.md	Phase 4 设计文档
OPTIC_QAT_README.md	QAT 技术文档
复赛-test.md	竞赛设计文档
log.md	原始训练日志 (所有 console 输出)
log_mixed.md	Mixed 训练日志
log_phase4_fixed.md	Phase4 修复版日志
log_optic_int8.md	INT8 容器推理日志 (2026-07-10)
log_optic_int4.md	INT4 容器推理日志 (2026-07-10)
log_optic_lsq.md	LSQ+ 容器推理日志 (2026-07-10)
log_optic_kd.md	KD+INT4 容器推理日志 (2026-07-10)
log_optic_mixed_model1.md	Model 1 Mixed 容器推理日志 (2026-07-10 ~ 11)
osimulator/GAZELLE_ARCHITECTURE.md	Gazelle 硬件逆向报告

15. 后续方向

15.1 短期 (本周)

1. ~~Model 3 int8+KD 训练~~  **已完成** (2026-07-12): 训练 92.35% (val), osimulator Quick 50/全量数字因 Bug #11 待复测
2. ~~Model 1 int8 训练~~  **已完成** (2026-07-13): 变体 A val **98.15%** / B val **98.02%**, 量化无损 ($\pm 0.06\%$)。详见 §11.11
3. ~~Model 3 v3 全量 osimulator 完成~~  **已完成** (2026-07-12): 数字因 Bug #11 待复测
4.  **修复 test 泄漏后重训** (Bug #11): split 已修 (§12), 需用三分划分重训 Model 1/2/3 以获得干净的独立 test 数, 复测历史 osimulator 精度

15.2 中期 (容器部署)

1. 选取最佳模型部署:
 - Model 1 Mixed (98.26% int4) — 精度最高, 但模型较大 (2.39M), 推理极慢 (~150s/张)
 - **Model 2 v3 int8 (93.28% osimulator) — 推荐首选:** 精度高, 模型小 (268K), 已完整验证
 - **Model 3 v3 int8+KD (93.26% osimulator 全量) — 强力候选:** KD 加持, 同等轻量, 已完整验证
2. 容器内完整验证:

```
# INT8 容器 osimulator 全量验证 (已完成 2026-07-10)
python optic_inference_int8.py # 5400 张独立测试集, 93.28%, ~4.1h

# Phase4 / Mixed 容器 (QAT 伪量化, 已完成)
python optic_inference_phase4.py
python optic_inference_mixed.py

# Optic 模式硬件仿真快速测试
python optic_inference_int8.py --quick 50 # 50 张, ~3min
```

3. 导出 ONNX/量化为实际 int8 整数: 将 QAT 权重导出为可直接部署的 int8 格式

15.3 长期 (持续优化)

1. **更大模型:** 扩宽 SpaceNet 通道 (16→32→64→32) 补偿量化损失
2. **渐进式量化:** 训练前半段 int8, 后半段 int4, 平滑过渡
3. **Distillation from FP32 teacher to int4 student:** 用 FP32 Model 1 做教师引导 int4 Model 2/3
4. **硬件在环训练:** 将 osimulator 实际输出作为训练信号, 端到端优化

15.4 精度路线图

当前最佳 → 短期目标 → 长期目标
Model 1: 98.26% (int4 Mixed) → 98.5% (int8) → 98.5%+
Model 2: 93.28% (osimulator) → 93.5% (int8+KD) → 94%
Model 3: 92.35% (int8+KD v3) →  93.26% (osimulator 全量) → 94%

16. INT4 osimulator 推理精度问题调查 (2026-07-11)

16.1 现象

optic_inference_int4.py (Model 2 v2 INT4, 训练精度 91.06%) 在容器 osimulator 上全量推理仅 ~88%,
比 QAT 伪量化模式 (94.57%) 低 6.6 个百分点.

QAT 模式: 94.57% (per-channel 输入, int4 权重, PyTorch 伪量化)
Optic 模式: 88.00% (per-tensor 输入, int8 权重, osimulator 真硬件)
差距: -6.57%

16.2 根因分析

Model 2 v2 INT4 训练配置与 osimulator 推理路径存在三处不可调和的不一致:

维度	QAT 训练 (optic_qat_v3)	Optic 推理 (osimulator)	兼容?
stem Conv	INT4 QAT (有量化噪声)	FP32 电子 (keep_first_conv_electronic=True)	✗
输入量化	per-channel signed int8	per-tensor unsigned uint8	✗
权重量化	int4 (16 级, scale=max/7)	int8 (256 级, scale=max/127)	✗

关键矛盾: 训练时 stem 是 QAT (对齐率 37.5% 太低不能走光计算),
推理时 stem 必须保持电子 FP32. 但 BN 统计量基于 QAT stem 输出分布训练,
推理时 FP32 stem 输出分布不同 → BN 偏移 → 级联误差.

对比 INT8 模型 (93.28% 成功): 训练时 first_conv_fp32=True → stem FP32 与推理一致,
这是精度对齐的关键.

尝试过的修复全部失败:

方案	结果	原因
weight_bit=4 (匹配 QAT)	18%	osimulator 内部 8a8w 编译, int4 权重值被误解释
per-channel + shift + correction	71%	correction term 量化误差被 shift×128 放大
BN 重校准 (batch_size=1)	40.5%	单样本 BN 统计量噪声过大
per-channel 预量化 + engine 重量化	87%	双重量化引入额外舍入误差
per-channel + scale 吸收 (干净方案)	88%	scale 吸收后 weight 重量化破坏了 QAT 训练的权重网格

16.3 结论

v2 INT4 模型 (全层 QAT, 无 stem FP32) 在 osimulator 上的实际上限是 ~88%.
与 QAT 模式的差距来自 stem 训练/推理不一致的根本性矛盾, 无法通过推理路径修复绕过.

16.4 新方案: Phase4 v4 — osimulator 兼容 INT4 训练

策略: 用 INT8 模型的训练配方 (first_conv_fp32=True) + INT4 权重, 使训练配置与 osimulator 推理路径天然对齐.

	v2 INT4 (旧, 不可部署)	v4 INT4 (新, osimulator 兼容)
stem	INT4 QAT	FP32 (匹配推理)
weight_bits	4	4
QAT 模块	optic_qat_v3	optic_qat_v4
训练噪声	STE (std=0.02*scale)	GazelleNoise (DAC 7.5+TIA)
推理 weight_bit	N/A	8 (osimulator 原生, int4→int8 升级无损)
推理 stem	N/A	电子 FP32 (匹配训练)
预期光学精度	88% (实际上限)	~90-92%

训练命令:

```
# Phase4 v3 脚本已支持 --wbits 4, 自动使用 first_conv_fp32=True
python model2_spacenet_v1_phase4_v3.py --wbits 4
# 权重: spacenet_v1_phase4_v3_int4.pth
```

推理脚本: optic_inference_int4_v2.py — 与 INT8 容器完全相同的 osimulator 配置 (input_bit=8, weight_bit=8, keep_first_conv_electronic=True), 仅权重文件不同.

16.5 v4 方案验证结果 (失败)

v4 方案训练完成后 (spacenet_v1_phase4_v3_int4.pth), 在容器内验证:

训练结果:

Int4 QAT (val):

91.94%

← 训练成功

Float32 (val):

82.80%

← Gazelle 噪声使权重过度依赖 int4 网络

容器验证 (test set, 200 images):

QAT int4 模式:

97.00%

← PyTorch 伪量化, per-channel

Optic osimulator:

84.00%

← 比 v2 的 88% 更差!

v4 比 v2 更差的原因: Gazelle 噪声 (DAC ENOB=7.5 + TIA) 使权重更"特化"于 int4 量化网络. Float32 精度从 v2 的 91% 降至 83%, 说明去量化后的权重已无法正常工作. 换到 int8 网络 (osimulator) 时偏差进一步放大.

核心结论: int4 训练的权重在 int8 推理路径上必然有损失. "int4→int8 升级无损" 的假设是**错误**的 — 量化网络 scale 不同 (max/7 vs max/127), 相同的浮点权重落在不同的量化值上.

16.6 最终结论

部署方案	光学精度	状态
INT8 v3 (spacenet_v1_phase4_v3_int8.pth)	93.28%	✅ 已验证, 推荐
INT4 v2 (spacenet_v1_phase4_v2_ste.pth)	~88%	⚠️ 可用, 比 QAT 低 ~6%
INT4 v4 (spacenet_v1_phase4_v3_int4.pth)	84%	❌ 不如 v2

INT4 在 osimulator 上 ~88% 的原因 (三个不可消除的误差源):

- 权重 int4→int8 重量化:** QAT 训练 scale=max/7 (16级), osimulator scale=max/127 (256级). 同一浮点权重在不同网络上取整不同, 每个权重 ~0.3% 相对误差, 全模型累积 ~3-4%.
- 激活 per-channel→per-tensor:** QAT 训练每个输入通道独立 scale, osimulator 的 im2col 展平后所有通道共享一个 scale. 通道间动态范围差异越大, 误差越大.
- stem QAT→FP32:** 训练时 stem 参与 int4 QAT (有量化噪声), 推理时 stem 保持 FP32, BN 统计量基于 QAT 分布, 应用于 FP32 输出时有轻微偏移.

三项叠加 → 总损失 ~6% (91% → 88%, 或 QAT 94% → 88%).

推荐部署策略: 直接使用 INT8 模型 (93.28%), 如需 INT4 则接受 88% 上限, 在报告中说明上述三个量化不对齐因素.

16.7 文件清单

文件	用途
optic_inference_int4.py	INT4 v2 容器推理 (~88%, 已文档化限制)
optic_inference_int4_v2.py	INT4 v4 容器推理 (实验性, 84%, 不推荐)
spacenet_v1_phase4_v2_ste.pth	INT4 v2 权重 (推荐用于 INT4 部署)
spacenet_v1_phase4_v3_int4.pth	INT4 v4 权重 (训练完成, osimulator 效果不佳)

16.8 Model 3 v3 int8+KD — int4 困境的解决方案 (2026-07-12) ★

§16.1-16.6 详细分析了 INT4 模型在 osimulator 上 ~88% 上限的三重根因。
Model 3 v3 的实践证明了解决方案的有效性:

	v2 int4 (旧方案)	v3 int8+KD (新方案)
训练 stem	int4 QAT	FP32 (first_conv_fp32=True)
训练权重	int4 (16级)	int8 (256级, 匹配 osimulator)
推理 stem	FP32 电子 (与训练矛盾)	FP32 电子 (与训练一致 ✓)
推理权重	int8 (与训练 int4 网格不对齐)	int8 (与训练一致 ✓)
osimulator 精度	84.33%	93.26% (全量 5400)
训练→osimulator 损失	-7.17%	+0.91%

核心原则: 训练配置必须与 osimulator 推理路径完全一致, 不能依赖"无损转换"假设。
int4→int8 网格转换并非无损 (max/7 → max/127), 必须在训练时就用 int8 量化。

适用于所有后续模型的 **checklist**:

- 1.  first_conv_fp32=True (训练) ↔ keep_first_conv_electronic=True (推理)
- 2.  weight_bits=8 (训练) ↔ weight_bit=8 (推理, osimulator 原生)
- 3.  Gazelle 噪声 (训练) ↔ osimulator 物理噪声 (推理)